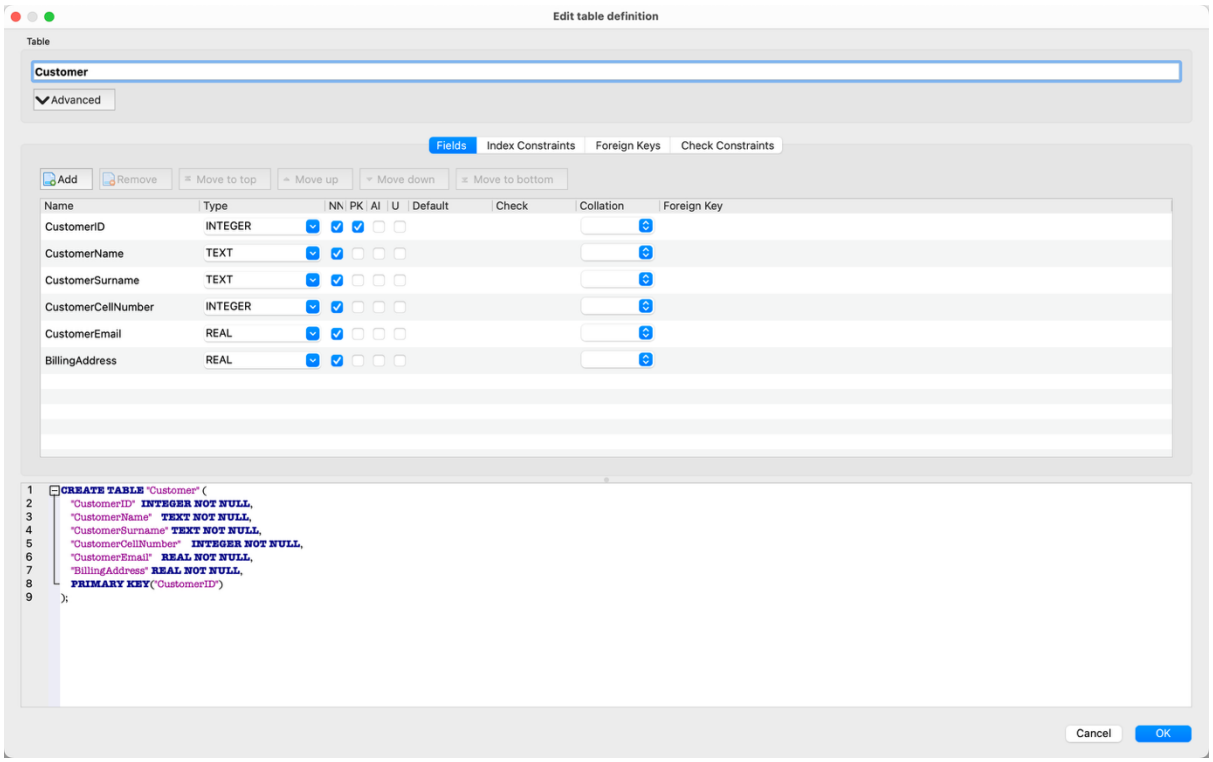
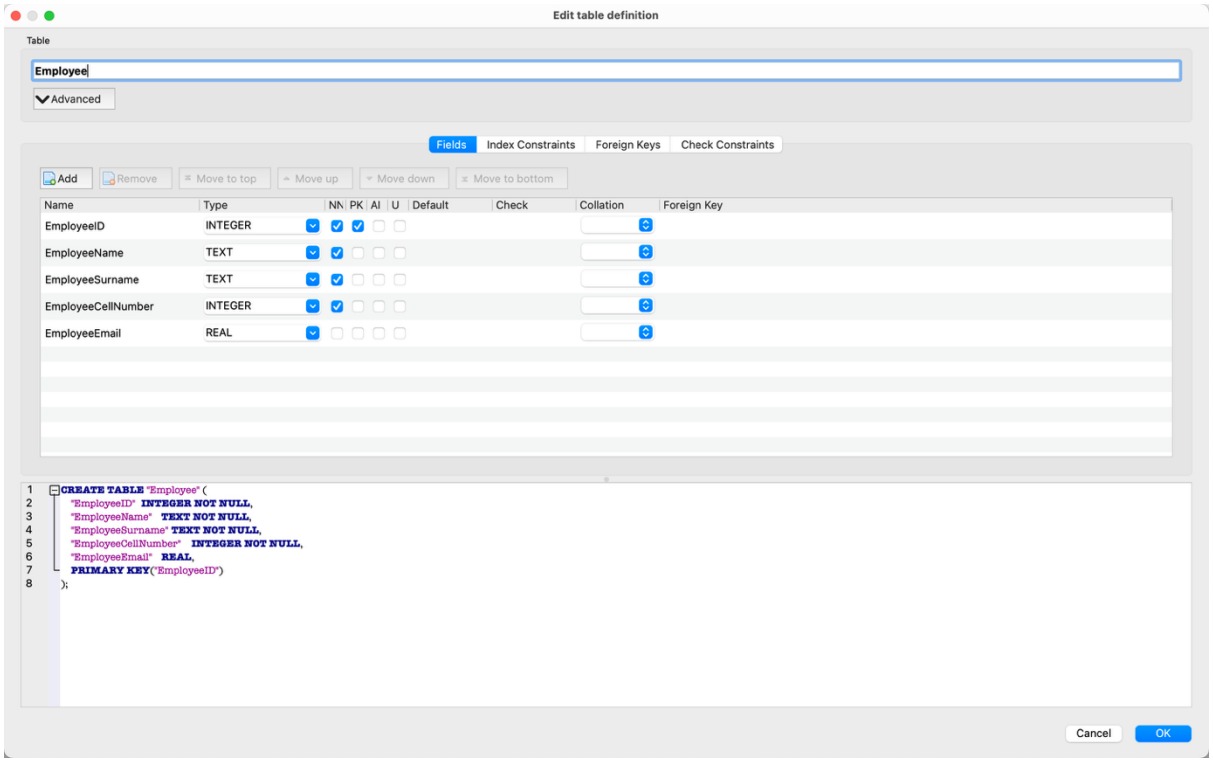
ITAPA FORMATIVE ASSESMENT

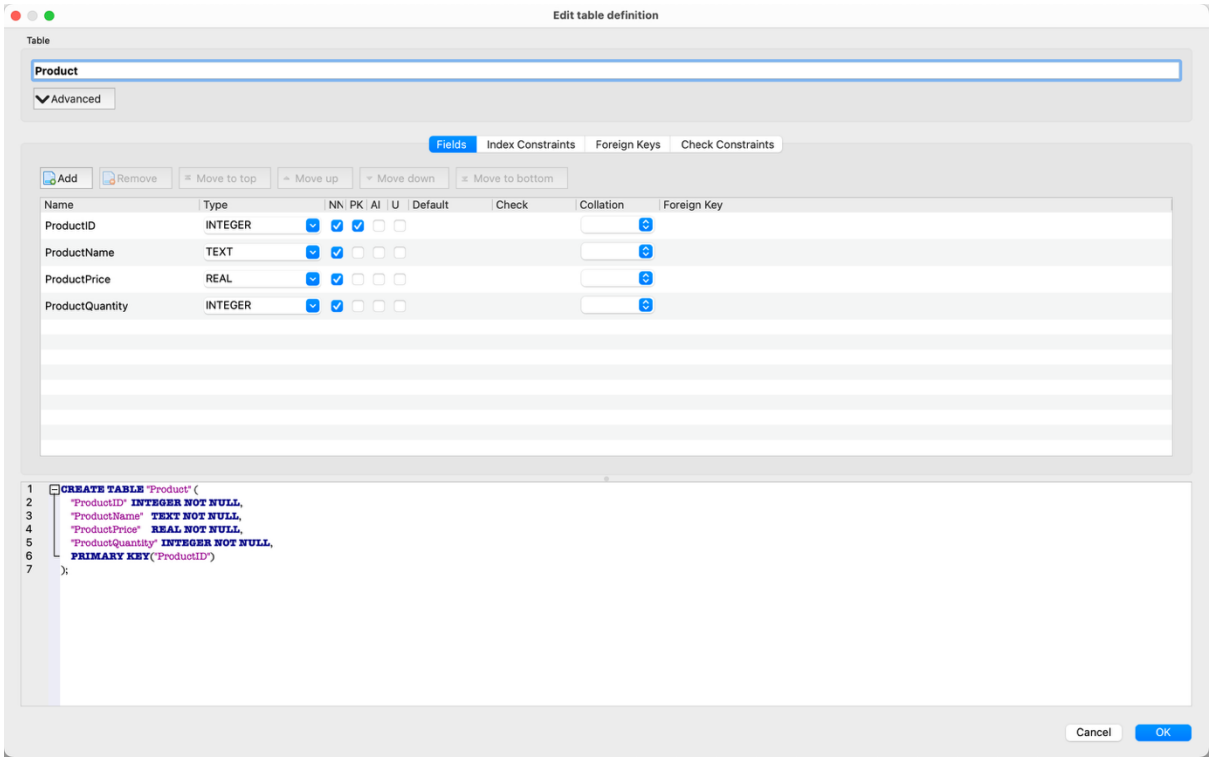
Sibusison Mfikwe

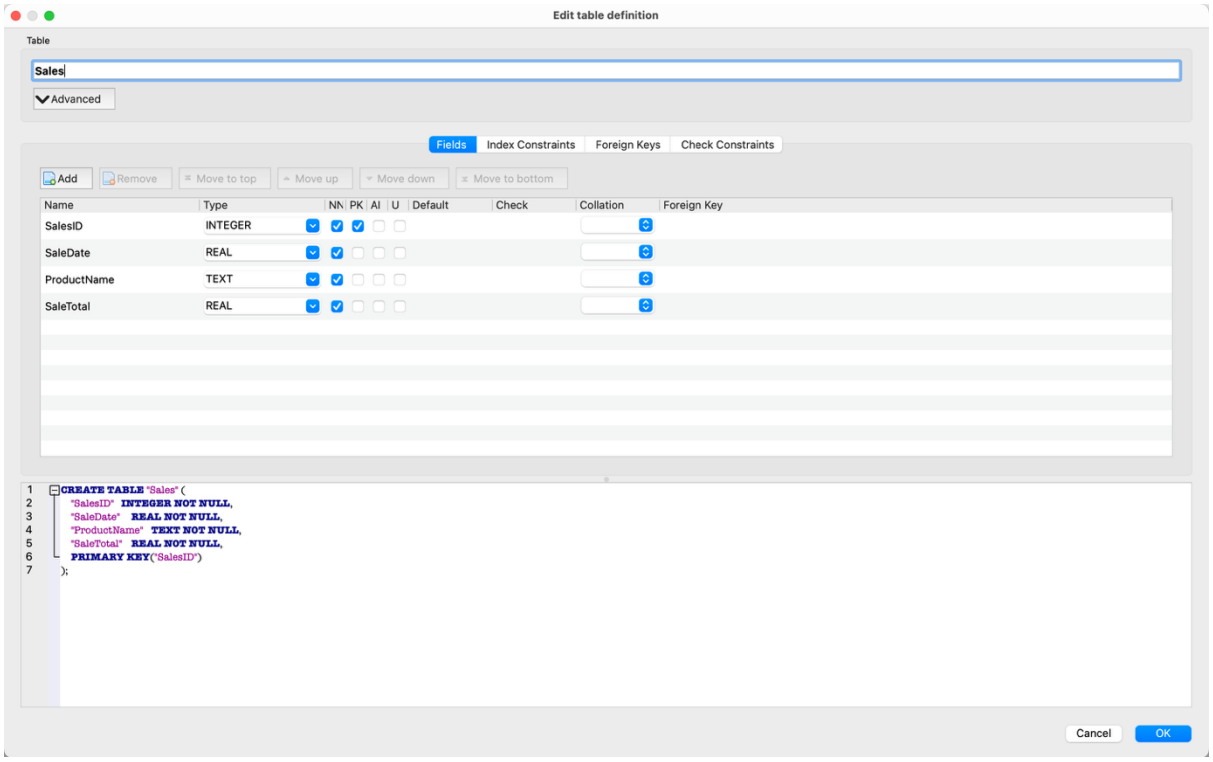
ITAPA GROUP 2 eduv5561348@vossie.net

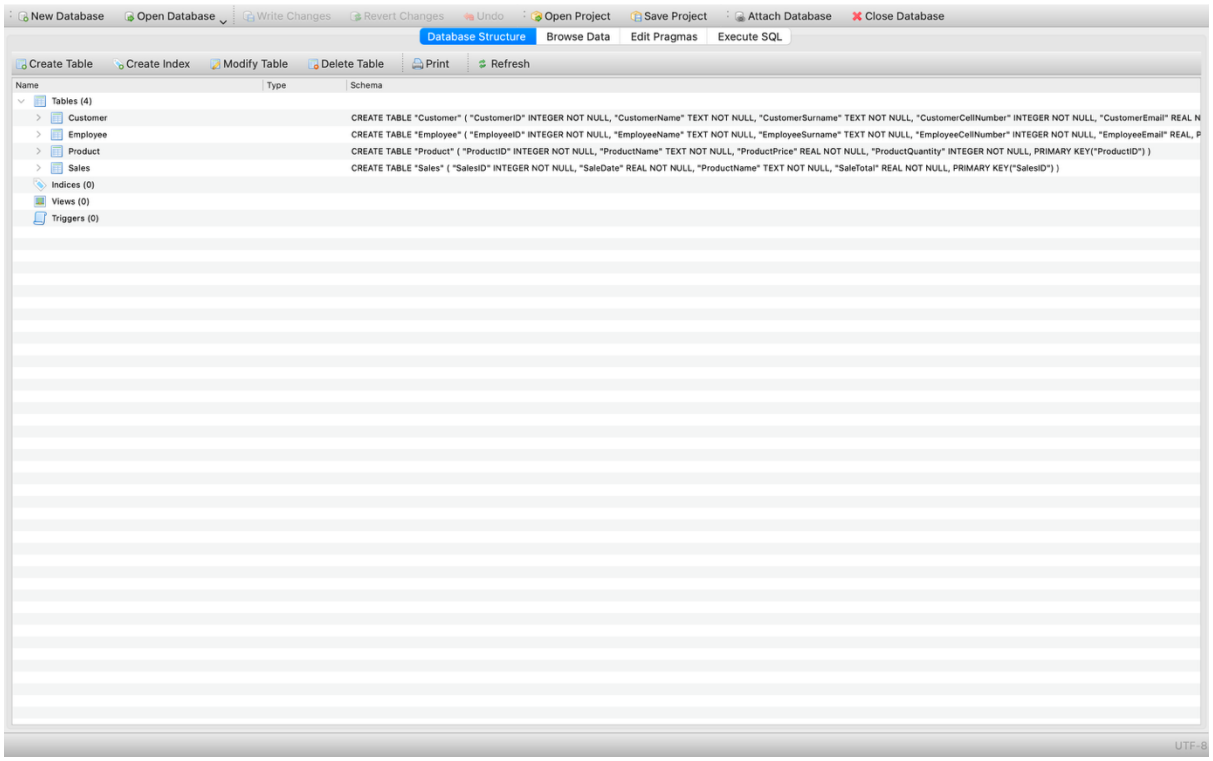
1.


Edit table definition
Table
Customer
Advanced
Fields
Index Constraints
Foreign Keys
Check Constraints
Add
Remove
Move to top
Move up
Move down
Move to bottom
Name
Type
NN
PK
AI
U
Default
Check
Collation
Foreign Key
CustomerID
INTEGER
CustomerName
TEXT
CustomerSurname
TEXT
CustomerCellNumber
INTEGER
CustomerEmail
REAL
BillingAddress
REAL
CREATE TABLE "Customer" (
"CustomerID" INTEGER NOT NULL,
"CustomerName" TEXT NOT NULL,
"CustomerSurname" TEXT NOT NULL,
"CustomerCellNumber" INTEGER NOT NULL,
"CustomerEmail" REAL NOT NULL,
"BillingAddress" REAL NOT NULL,
PRIMARY KEY("CustomerID")
);
Cancel
OK


Edit table definition
Table
Employee
Advanced
Fields
Index Constraints
Foreign Keys
Check Constraints
Add
Remove
Move to top
Move up
Move down
Move to bottom
Name
Type
NN
PK
AI
U
Default
Check
Collation
Foreign Key
EmployeeID
INTEGER
EmployeeName
TEXT
EmployeeSurname
TEXT
EmployeeCellNumber
INTEGER
EmployeeEmail
REAL
CREATE TABLE "Employee" (
"EmployeeID" INTEGER NOT NULL,
"EmployeeName" TEXT NOT NULL,
"EmployeeSurname" TEXT NOT NULL,
"EmployeeCellNumber" INTEGER NOT NULL,
"EmployeeEmail" REAL,
PRIMARY KEY("EmployeeID")
);
Cancel
OK


Edit table definition
Table
Product
Advanced
Fields
Index Constraints
Foreign Keys
Check Constraints
Add
Remove
Move to top
Move up
Move down
Move to bottom
Name
Type
NN
PK
AI
U
Default
Check
Collation
Foreign Key
ProductID
INTEGER
ProductName
TEXT
ProductPrice
REAL
ProductQuantity
INTEGER
1 CREATE TABLE "Product" (
2 "ProductID" INTEGER NOT NULL,
3 "ProductName" TEXT NOT NULL,
4 "ProductPrice" REAL NOT NULL,
5 "ProductQuantity" INTEGER NOT NULL,
6 PRIMARY KEY("ProductID")
7);
Cancel
OK


Edit table definition
Table
Sales
Advanced
Fields
Index Constraints
Foreign Keys
Check Constraints
Add
Remove
Move to top
Move up
Move down
Move to bottom
Name
Type
NN
PK
AI
U
Default
Check
Collation
Foreign Key
SalesID
INTEGER
SaleDate
REAL
ProductName
TEXT
SaleTotal
REAL
1 CREATE TABLE "Sales" (
2 "SalesID" INTEGER NOT NULL,
3 "SaleDate" REAL NOT NULL,
4 "ProductName" TEXT NOT NULL,
5 "SaleTotal" REAL NOT NULL,
6 PRIMARY KEY("SalesID")
7);
Cancel
OK


New Database
Open Database
Write Changes
Revert Changes
Undo
Open Project
Save Project
Attach Database
Close Database
Database Structure
Browse Data
Edit Pragmas
Execute SQL
Create Table
Create Index
Modify Table
Delete Table
Print
Refresh
Name
Type
Schema
Tables (4)
Customer
CREATE TABLE "Customer" ("CustomerID" INTEGER NOT NULL, "CustomerName" TEXT NOT NULL, "CustomerSurname" TEXT NOT NULL, "CustomerCellNumber" INTEGER NOT NULL, "CustomerEmail" REAL N
Employee
CREATE TABLE "Employee" ("EmployeeID" INTEGER NOT NULL, "EmployeeName" TEXT NOT NULL, "EmployeeSurname" TEXT NOT NULL, "EmployeeCellNumber" INTEGER NOT NULL, "EmployeeEmail" REAL, P
Product
CREATE TABLE "Product" ("ProductID" INTEGER NOT NULL, "ProductName" TEXT NOT NULL, "ProductPrice" REAL NOT NULL, "ProductQuantity" INTEGER NOT NULL, PRIMARY KEY("ProductID"))
Sales
CREATE TABLE "Sales" ("SalesID" INTEGER NOT NULL, "SaleDate" REAL NOT NULL, "ProductName" TEXT NOT NULL, "SaleTotal" REAL NOT NULL, PRIMARY KEY("SalesID"))
Indices (0)
Views (0)
Triggers (0)
UTF-8

2.

```
from abc import ABC, abstractmethod
class PersonManager(ABC):
    def __init__(self, name, surname, cell_number, email):
        self.name = name
        self.surname = surname
        self.cell_number = cell_number
        self.email = email


@abstractmethod
    def insert_person(self):
        pass

    @abstractmethod
    def remove_person(self):
        pass

    @abstractmethod
    def display_all(self):
        pass
```

Proof of inheritance

```
class Employee(PersonManager):
```

```
class Customer(PersonManager):
```

Implementation of Abstract Methods

```
emp1 = Employee("John", "Mafela", "0624316734", "john@gmail.com")
emp1.insert_person()
emp1.display_all()

cust1 = Customer("Martin", "Thobe", "0633271234", "martin@gmail.com", " 25 Mosala
Street")
cust1.insert_person()
cust1.display_all()
```

3. EmployeeManager class

```
class EmployeeManager(PersonManager):

    def insert_person(self):
        conn = connect_db()
        cursor = conn.cursor()
```

```
        cursor.execute("""
        INSERT INTO Employee (name, surname, cell_number, email)
        VALUES (?, ?, ?, ?)
        """, (self.name, self.surname, self.cell_number, self.email))

        conn.commit()
        conn.close()

    def remove_person(self):
        conn = connect_db()
        cursor = conn.cursor()

        cursor.execute("DELETE FROM Employee WHERE email = ?", (email,))
        conn.commit()
        conn.close()

    def display_all(self):
        conn = connect_db()
        cursor = conn.cursor()

        cursor.execute("SELECT * FROM Employee")
        employees = cursor.fetchall()

        for emp in employees:
            print(emp)

        conn.close()
```

4. CustomerManger class

```
class CustomerManager(PersonManager):

    def __init__(self, name, surname, cell_number, email, billing_address):
        super().__init__(name, surname, cell_number, email)
        self.billing_address = billing_address

    def insert_person(self):
        conn = connect_db()
        cursor = conn.cursor()

        cursor.execute("""
        INSERT INTO Customer (name, surname, cell_number, email,
billing_address)
        VALUES (?, ?, ?, ?, ?)
       """, (self.name, self.surname, self.cell_number, self.email,
self.billing_address))

        conn.commit()
        conn.close()
        print("Customer added successfully")
```

```
    def remove_person(self):
        conn = connect_db()
        cursor = conn.cursor()

        email = input()

        cursor.execute("DELETE FROM Customer WHERE email = ?", (email,))
        conn.commit()
        conn.close()
        print("Customer removed")

    def display_all(self):
        conn = connect_db()
        cursor = conn.cursor()

        cursor.execute("SELECT * FROM Customer")
        customers = cursor.fetchall()

        for cust in customers:
            print(cust)

       conn.close()
```

5.

```
port sqlite3
from abc import ABC, abstractmethod
from datetime import datetime

def connect_db():
    return sqlite3.connect("london_roots.db")

class PersonManager(ABC):
    def __init__(self, name, surname, cell_number, email):
        self.name = name
        self.surname = surname
        self.cell_number = cell_number
        self.email = email

    @abstractmethod
    def insert_person(self):
        pass

    @abstractmethod
    def remove_person(self):
        pass

    @abstractmethod
   def display_all(self):
```

```
        pass

class EmployeeManager(PersonManager):
    def insert_person(self):
        conn = connect_db()
        cursor = conn.cursor()
     cursor.execute("SELECT * FROM Employee WHERE email = ?",(self.email,))
        if cursor.fetchone():
            print("Employee already exists")
            conn.close()
            return

        cursor.execute("""
        INSERT INTO Employee (name, surname, cell_number, email)
        VALUES (?, ?, ?, ?)
        """, (self.name, self.surname, self.cell_number, self.email))
        conn.commit()
        conn.close()
        print("Employee added successfully")

    def remove_person(self):
        conn = connect_db()
        cursor = conn.cursor()
        email = input("Enter employee email to remove: ")
        cursor.execute("DELETE FROM Employee WHERE email = ?", (email,))
        conn.commit()
        conn.close()
        print("Employee removed")

    def display_all(self):
        conn = connect_db()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM Employee")
        employees = cursor.fetchall()
        for emp in employees:
            print(emp)
        conn.close()

class Customer(PersonManager):
    def __init__(self, name, surname, cell_number, email, billing_address):
        super().__init__(name, surname, cell_number, email)
        self.billing_address = billing_address

    def insert_person(self):
        conn = connect_db()
        cursor = conn.cursor()
     cursor.execute("SELECT * FROM Customer WHERE email = ?",(self.email,))
        if cursor.fetchone():
           print("Customer already exists")
            conn.close()
            return
```

```
        cursor.execute("""
  INSERT INTO Customer (name, surname, cell_number, email,billing_address)
        VALUES (?, ?, ?, ?, ?)
""", (self.name, self.surname, self.cell_number,
self.email,self.billing_address))
        conn.commit()     conn.close()
        print("Customer added successfully")
    def remove_person(self):
        conn = connect_db()
        cursor = conn.cursor()
        email = input("Enter customer email to remove: ")
        cursor.execute("DELETE FROM Customer WHERE email = ?", (email,))
        conn.commit()
        conn.close()     print("Customer removed")

    def display_all(self):
        conn = connect_db()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM Customer")
        customers = cursor.fetchall()
        for cust in customers:
            print(cust)
        conn.close()

class Store:
    def addProduct(self, name, price, quantity):
        conn = connect_db()
cursor = conn.cursor()
        cursor.execute("SELECT * FROM Product WHERE product_name ?",(name,))
        if cursor.fetchone():
            print("Product already exists")
            conn.close()
            return
        cursor.execute("""
        INSERT INTO Product (product_name, product_price, quantity)
        VALUES (?, ?, ?)
        """, (name, price, quantity))
        conn.commit()
        conn.close()
        print("Product added successfully")

    def removeProduct(self, product_id):
        conn = connect_db()
        cursor = conn.cursor()
        cursor.execute("DELETE FROM Product WHERE product_id = ?",
(product_id,))
        conn.commit()
        conn.close()
        print("Product removed")
```

```
    def updateProduct(self, product_id, name=None, price=None,
quantity=None):
        conn = connect_db()
        cursor = conn.cursor()
    if name:
            cursor.execute("UPDATE Product SET product_name = ? WHERE
product_id = ?", (name, product_id))
        if price is not None:
            cursor.execute("UPDATE Product SET product_price = ? WHERE
product_id = ?", (price, product_id))
        if quantity is not None:
            cursor.execute("UPDATE Product SET quantity = ? WHEREproduct_id
= ?", (quantity, product_id))
        conn.commit()
        conn.close() print("Product updated successfully")

    def displayProducts(self):
        conn = connect_db()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM Product")
        products = cursor.fetchall()
        for prod in products:
        print(prod)
        conn.close()
    def sellProduct(self, product_id, quantity_to_sell):
        conn = connect_db()
        cursor = conn.cursor()
       cursor.execute("SELECT product_name, product_price, quantityFROM
Product WHERE product_id = ?", (product_id,))
        product = cursor.fetchone()
        if product is None:
            print("Product not found")
            conn.close()
            return
        product_name, product_price, current_quantity = product
        if current_quantity == 0:
            print("Cannot sell this product, quantity is 0")
            conn.close()
            return
        if quantity_to_sell > current_quantity:
            print(f"Cannot sell {quantity_to_sell}, only{current_quantity}
available")
            conn.close()
            return
        new_quantity = current_quantity - quantity_to_sell
        cursor.execute("UPDATE Product SET quantity = ? WHERE product_id =
?", (new_quantity, product_id))
        sale_total = product_price * quantity_to_sell
        sale_date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        cursor.execute("""
        INSERT INTO Sales (sale_date, product_name, sales_total)
```

```
        VALUES (?, ?, ?)
        """, (sale_date, product_name, sale_total))
        conn.commit()
        conn.close()
        print(f"Sold {quantity_to_sell} of {product_name}. Total:
{sale_total}")

    def displaySales(self):
        conn = connect_db()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM Sales")
        sales = cursor.fetchall()
        for sale in sales:
            print(sale)
        conn.close()

emp1 = EmployeeManager("John", "Mafela", "06342534175","John@gmail.com")
emp1.insert_person()
emp1.display_all()

cust1 = Customer("", "Thobz", "0987654321", "mmotla@email.com","18 Main
Street")
cust1.insert_person()
cust1.display_all()

store = Store()
store.addProduct("Eco Bottle", 15.50, 10)
store.addProduct("Reusable Bag", 5.75, 20)
store.displayProducts()
store.sellProduct(1, 3)
store.displaySales()
```